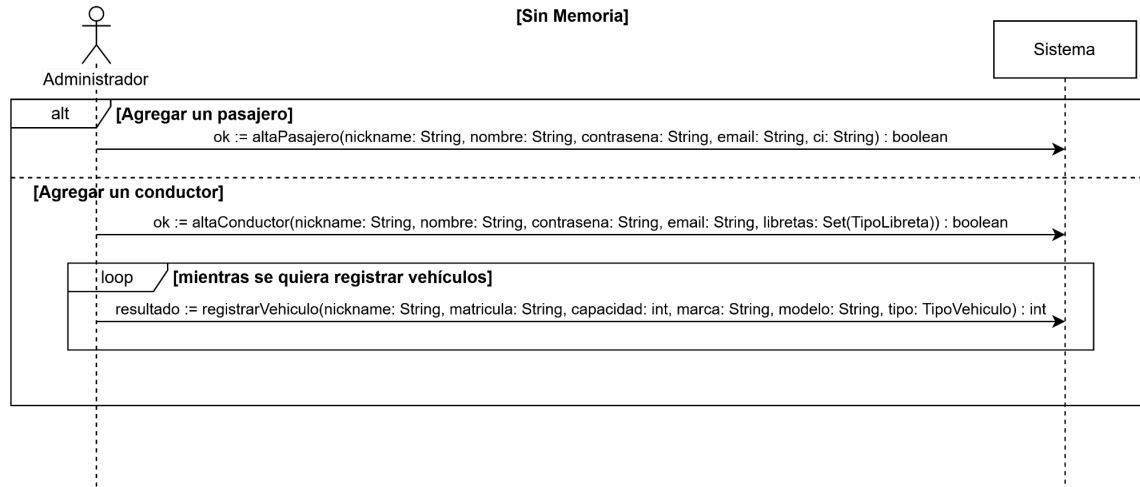


Programación 4

Informe del Modelo de Casos de Uso

ALTA DE USUARIO

Diagramas de Secuencia del Sistema



Contratos

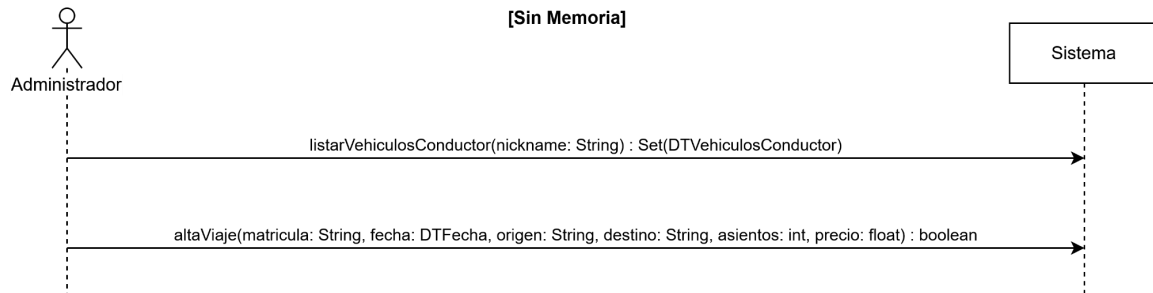
Operación	<code>altaPasajero(nickname: String, nombre: String, contrasena: String, email: String, ci: String) : boolean</code>
Entrada	<code>nickname</code> : Nickname del pasajero que se creará. <code>nombre</code> : Nombre del pasajero. <code>contrasena</code> : Contraseña del pasajero. <code>email</code> : Dirección de correo electrónico del pasajero. <code>ci</code> : Número de documento del pasajero.
Salida	Booleano indicando si la operación fue exitosa
Descripción	Crear un nuevo pasajero con los datos proporcionados. Notifica en caso de éxito o de error al crear el pasajero.
Precondiciones	La <code>contrasena</code> tiene al menos 8 caracteres.
Postcondiciones	- Si existe un Usuario <code>u</code> tal que <code>u.nickname = nickname</code> se retorna <code>false</code> - Si no existe un Usuario <code>u</code> tal que <code>u.nickname = nickname</code> - Se crea una instancia de Pasajero <code>p</code> con <code>p.nickname = nickname</code>, <code>p.nombre = nombre</code>, <code>p.contrasena = contrasena</code>, <code>p.email = email</code> y <code>p.ci = ci</code>. - Se retorna <code>true</code>

Operación	<code>altaConductor(nickname: String, nombre: String, contraseña: String, email: String, libretas: Set(TipoLibreta)) : boolean</code>
Entrada	<code>nickname</code>: Nickname del conductor que se creará. <code>nombre</code>: Nombre del conductor. <code>contrasena</code>: Contraseña del conductor. <code>email</code>: Dirección de correo electrónico. <code>libretas</code>: Conjunto de libretas del conductor.
Salida	Booleano indicando si la operación fue exitosa
Descripción	Crear un nuevo conductor con los datos proporcionados. Notifica en caso de éxito o de error al crear el conductor.
Precondiciones	• La <code>contrasena</code> tiene al menos 8 caracteres.
Postcondiciones	• Si existe un Usuario <code>u</code> tal que <code>u.nickname = nickname</code> se retorna <code>false</code> • Si no existe un Usuario <code>u</code> tal que <code>u.nickname = nickname</code> ◦ Se crea una instancia de Conductor <code>c</code> con <code>c.nickname = nickname</code>, <code>c.nombre = nombre</code>, <code>c.contrasena = contraseña</code>, <code>c.email = email</code> y <code>c.libretas = libretas</code>. ◦ Se retorna <code>true</code>

Operación	registrarVehiculo(nickname: String, matricula: String, capacidad: int, marca: String, modelo: String, tipo: TipoVehiculo) : int
Entrada	nickname: Nickname del conductor. matricula: Matrícula del vehículo a crear. capacidad: Cantidad máxima de asientos del vehículo. marca: Marca del vehículo.. modelo: Modelo del vehículo. tipo: tipo de vehículo (auto o moto).
Salida	int indicando si la operación fue exitosa o que error ocurrió (0 correcta, -1 matrícula repetida, -2 no tiene tipo de licencia para el vehículo)
Descripción	Crear un vehículo nuevo con los datos proporcionados y lo asocia al conductor. Notifica en caso de éxito o de error al crear el vehículo.
Precondiciones	• Existe un conductor <i>c</i> con <i>c.nickname</i> = <i>nickname</i>.
Postcondiciones	• Si existe un Vehículo <i>v</i> tal que <i>v.matricula</i> = <i>matricula</i> se retorna -1 • Si <i>tipo</i> = Auto y <i>c.libretas</i> no contiene Auto Profesional o Auto Amateur se retorna -2 • Si <i>tipo</i> = Moto y <i>c.libretas</i> no contiene Moto Profesional o Moto Amateur se retorna -2 • Si no se cumple lo anterior: ○ Se crea una instancia de Vehículo <i>v</i> con <i>v.matricula</i> = <i>matricula</i>, <i>v.capacidad</i> = <i>capacidad</i>, <i>v.marca</i> = <i>marca</i>, <i>v.modelo</i> = <i>modelo</i> y <i>v.tipo</i> = <i>tipo</i>. ○ Se crea un link entre la instancia de Conductor <i>c</i> tal que <i>c.nickname</i> = <i>nickname</i> y el Vehículo <i>v</i> ○ Se retorna 0

ALTA DE VIAJE

Diagramas de Secuencia del Sistema



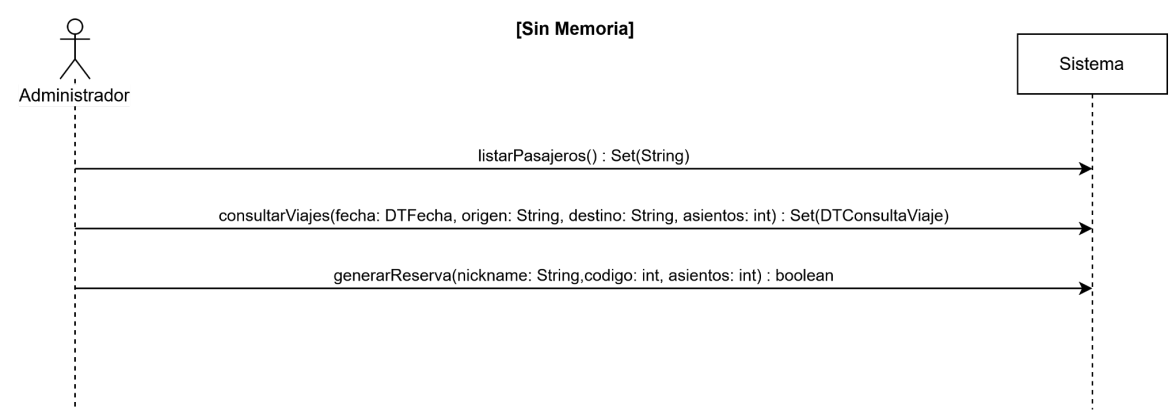
Contratos

Operación	<code>listarVehiculosConductor(nickname: String) : Set(DTVehiculosConductor)</code>
Entrada	<code>nickname</code> : Nickname del conductor
Salida	Conjunto de <code>DTVehiculosConductor</code> de todos los vehículos registrados para el conductor
Descripción	Devuelve la información de matrícula, modelo y capacidad para todos los vehículos registrados para el conductor.
Precondiciones	Existe una instancia de <code>Conductor c</code> con <code>nickname = nickname</code>
Postcondiciones	Se retorna el conjunto de los <code>datavalue dtv</code> del tipo <code>DTVehiculosConductor</code> correspondientes a las instancias de <code>Vehículo v</code> que estén relacionados con el <code>Conductor c</code> con <code>c.nickname = nickname</code> mediante <code>Registra</code> tal que <code>dtv.matricula = v.matricula</code>, <code>dtv.modelo = v.modelo</code> y <code>dtv.capacidad = v.capacidad</code>

Operación	<code>altaViaje(matricula: String, fecha: DTFecha, origen: String, destino: String, asientos: int, precio: float) : boolean</code>
Entrada	<code>matricula</code>: Matrícula del vehículo. <code>fecha</code>: Fecha de realización del viaje. <code>origen</code>: Texto del origen del viaje <code>destino</code>: Texto del destino del viaje. <code>asientos</code>: Cantidad de asientos disponibles para reservar. <code>precio</code>: Precio por asiento al momento de la reserva.
Salida	Booleano indicando si la operación fue exitosa.
Descripción	Crear un nuevo viaje en el sistema con los datos proporcionados relacionandolo con el vehículo y notificar si se pudo crear con éxito.
Precondiciones	● Existe una instancia de Vehículo <code>v</code> con <code>v.matricula = matricula</code>
Postcondiciones	● En caso de que <code>asientos</code> sea mayor que <code>v.capacidad</code> se retorna <code>false</code> ● En caso de que exista un Viaje <code>vi</code> relacionado a un Vehículo <code>vc</code> relacionado al Conductor <code>c</code> relacionado al Vehículo <code>v</code> tal que <code>vi.fecha = fecha</code> se retorna <code>false</code> ● En caso contrario: ○ Se crea una instancia de Viaje <code>vi</code> con <code>vi.codigo = ultimoViaje + 1</code>, <code>vi.fecha = fecha</code>, <code>vi.origen = origen</code>, <code>vi.destino = destino</code>, <code>vi.asientosPublicados = asientos</code> y <code>vi.precio = precio</code>. ○ Se crea un link entre el Viaje <code>vi</code> y la instancia de Vehículo <code>v</code> tal que <code>v.matricula = matricula</code>. ○ Se actualiza <code>ultimoViaje</code> con <code>vi.codigo</code> ○ Se retorna <code>true</code>

GENERAR RESERVA

Diagramas de Secuencia del Sistema



Contratos

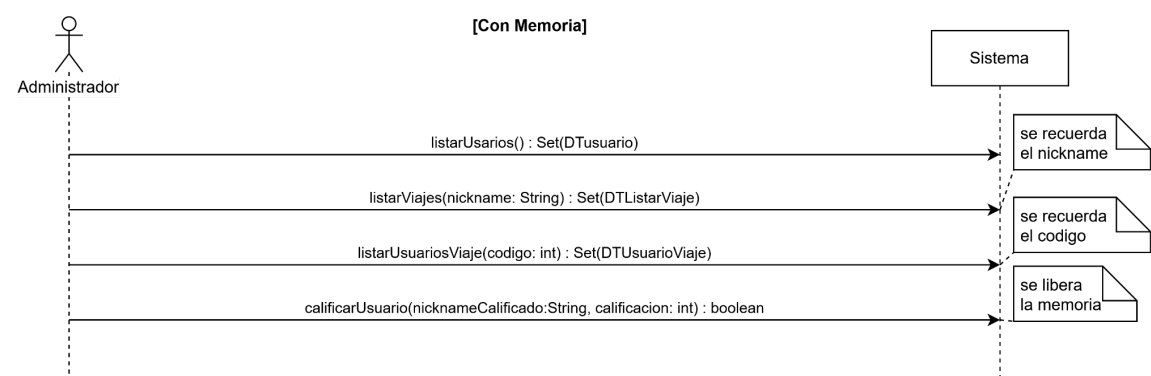
Operación	listarPasajeros() : Set(String)
Entrada	-
Salida	Conjunto de los nicknames de los pasajeros
Descripción	Devuelve los nicknames de todos los pasajeros.
Precondiciones	-
Postcondiciones	Se retorna el conjunto de los strings <code>p.nickname</code> correspondientes a todas las instancias de Pasajero <code>p</code>

Operación	<code>consultarViajes (fecha: DTFecha, origen: String, destino: String, asientos: int) : Set (DTConsultaViaje)</code>
Entrada	fecha: Fecha de los viajes a consultar. origen: Texto del origen de los viajes a consultar. destino: Texto del destino de los viajes a consultar. asientos: Cantidad de asientos a reservar.
Salida	Conjunto de los DTConsultaViaje de los viajes que cumple con los criterios de búsqueda
Descripción	Devuelve la información de los viajes que cumplen con las condiciones de búsqueda con la información del conductor y vehículo del mismo.
Precondiciones	-
Postcondiciones	Se retorna el conjunto de los datavalue dtcv del tipo DTConsultaViaje ordenados de menor a mayor dtcv.precioTotal y de mayor a menor dtcv.calificacionProm correspondientes a las instancias de Viaje vi que cumplan con vi.fecha = fecha, vi.origen = origen y vi.destino = destino y que la suma de r.asientosResevados de las Reservas r relacionadas al Viaje más asientos pasado por parámetro no supere vi.asientosPublicados, tomando el Vehículo v relacionado con el viaje y c el Conductor relacionado al Vehículo v el datavalue dtcv se construye de la forma dtcv.codigo = vi.codigo, dtcv.marca = v.marca, dtcv.modelo = v.modelo, dtcv.conductor = c.nombre, dtcv.calificacionProm = c.calificacionPromedio siendo y dtcv.precioTotal = vi.precio * asientos

Operación	<code>generarReserva (nickname: String, codigo: int, asientos: int) : boolean</code>
Entrada	nickname: nickname del usuario que quiere reservar. codigo: código del viaje a reservar. asientos: Cantidad de asientos a reservar.
Salida	Booleano indicando si la operación fue exitosa.
Descripción	Crea una reserva para el pasajero y el viaje pasado por parámetro.
Precondiciones	- Existe una instancia de Viaje vi con vi.codigo = codigo - Existe una instancia de Pasajero p con p.nickname = nickname
Postcondiciones	- Si el Viaje vi cumple con que la suma de r.asientosResevados de las Reservas r relacionadas al Viaje más asientos pasado por parámetro no supere vi.asientosPublicados y no existe una Reserva r relacionada con el Pasajero p y Viaje v - Se crea una instancia de Reserva r con r.asientosResevados = asientos y r.fecha = fechaActual - Se crea un link entre la Reserva r y Pasajero p - Se crea un link entre la Reserva r y Viaje vi - Se retorna true - En caso contrario se retorna false

CALIFICAR USUARIO

Diagramas de Secuencia del Sistema



Contratos

Operación	listarUsuarios() : Set (DTUsuario)
Entrada	-
Salida	Conjunto de DTUsuario de todos los usuarios
Descripción	Devuelve la información de nickname y nombre de todos los usuarios
Precondiciones	-
Postcondiciones	Se retorna el conjunto de los datavalue dtu del tipo DTUsuario correspondientes a las instancias de Usuario u donde dtu.nickname = u.nickname y dtu.nombre = u.nombre

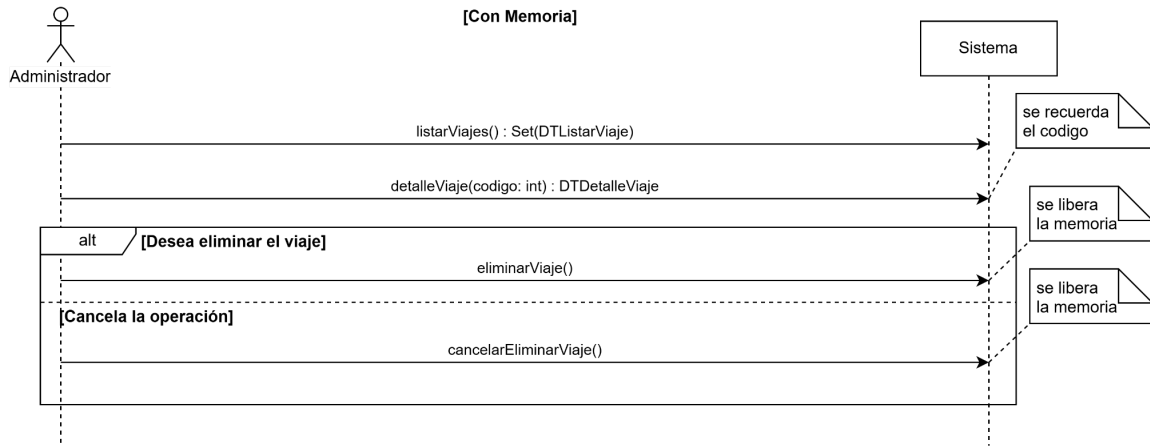
Operación	<code>listarViajes(nickname: String):Set(DTListarViaje)</code>
Entrada	<code>nickname</code> : nickname del usuario que quiere reservar.
Salida	Conjunto de todos DTViaje de viajes del usuario
Descripción	Devuelve la información de viajes relacionados al usuario pasado
Precondiciones	Existe una instancia de Usuario <code>u</code> con <code>u.nickname = nickname</code>
Postcondiciones	- Si el Usuario <code>u</code> es Pasajero - Se retorna el conjunto de los datavalue <code>dtvi</code> del tipo <code>DTListarViaje</code> correspondientes a las instancias de Viaje <code>vi</code> relacionados a cada Reserva <code>r</code> tal que <code>r</code> está relacionada al Pasajero <code>u</code> donde <code>dtvi.codigo = vi.codigo</code>, <code>dtvi.fecha = vi.fecha</code>, <code>dtvi.origen = vi.origen</code>, <code>dtvi.destino = vi.destino</code> y <code>dtvi.conductor = c.nickname</code> siendo <code>c</code> la instancia de Conductor relacionada al Vehículo <code>v</code> relacionado al Viaje <code>vi</code> - Si el Usuario <code>u</code> es Conductor - Se retorna el conjunto de los datavalue <code>dtvi</code> del tipo <code>DTListarViaje</code> correspondientes a las instancias de Viaje <code>vi</code> relacionados a un Vehículo <code>v</code> tal que <code>v</code> esté relacionado al Conductor <code>u</code> donde <code>dtvi.codigo = vi.codigo</code>, <code>dtvi.fecha = vi.fecha</code>, <code>dtvi.origen = vi.origen</code>, <code>dtvi.destino = vi.destino</code> y <code>dtvi.conductor = c.nickname</code> siendo <code>c</code> la instancia de Conductor relacionada al Vehículo <code>v</code> relacionado al Viaje <code>vi</code> - Se guarda en memoria el <code>nickname</code> pasado por parámetro

Operación	<code>listarUsuariosViaje(codigo: int):Set(DTUsuarioViaje)</code>
Entrada	<code>codigo</code> : código del viaje a obtener los usuarios.
Salida	Conjunto de los DTUsuarioViaje de los usuarios con reservas para el viaje y el conductor si corresponde
Descripción	Devuelve la información de los nicknames y nombre de todos los usuarios relacionados al viaje
Precondiciones	- Existe una instancia de Viaje <code>vi</code> con <code>vi.codigo = codigo</code> - Existe en memoria el <code>nickname</code>
Postcondiciones	- Se devuelve el conjunto de datavalue <code>dtuv</code> para cada instancia de Usuario <code>u</code> tal que <code>u.nickname ≠ nickname</code> y el Viaje <code>vi</code> está relacionado a una Reserva <code>r</code> relacionada al Usuario <code>u</code>, en cuyo caso el <code>dtuv.nickname = u.nickname</code> y <code>dtuv.tipo = Pasajero</code> o el Viaje <code>vi</code> está relacionado al Vehículo <code>v</code> relacionado al Usuario <code>u</code> en cuyo caso <code>dtuv.nickname = u.nickname</code> y <code>dtuv.tipo = Conductor</code> - Se guarda en memoria el pasado <code>codigo</code> por parámetro

Operación	<code>calificarUsuario(nicknameCalificado:String, calificacion: int) : boolean</code>
Entrada	<code>nicknameCalificado</code>: nickname del usuario a calificar <code>calificacion</code>: valor de la calificación para el usuario
Salida	Booleano indicando si la operación fue exitosa.
Descripción	Genera una nueva calificación para el usuario pasado por parámetro desde el usuario guardado en memoria para el viaje guardado en memoria
Precondiciones	• Existe en memoria el <code>nickname</code> del usuario calificador • Existe en memoria el <code>codigo</code> del viaje • El valor de <code>calificacion</code> es un valor entre 1 y 5 • Existe un <code>Usuario u_calificado</code> con <code>u_calificado.nickname = nicknameCalificado</code> • Existe una instancia de <code>Viaje vi</code> con <code>vi.codigo = codigo</code> • Existe una instancia de <code>Usuario u</code> con <code>u.nickname = nickname</code> • <code>u.nickname ≠ u_calificado.nickname</code>
Postcondiciones	• Si existe una instancia de <code>Calificacion c</code> tal que <code>c</code> está relacionada mediante <code>realiza</code> al <code>Usuario u</code>, califica al <code>Usuario u_calificado</code> y a una <code>Reserva r</code> tal que <code>r</code> está relacionado al <code>Viaje vi</code> se retorna <code>false</code> • En caso contrario: ◦ Se crea una instancia de <code>Calificacion c</code> tal que <code>c.calificacion = calificacion</code> y <code>c.fecha = fechaActual</code>. ◦ Se crea el link <code>realiza</code> entre la <code>Calificacion c</code> y <code>Usuario u</code> ◦ Se crea el link <code>califica</code> entre la <code>Calificacion c</code> y <code>Usuario u_calificado</code> ◦ Si el <code>Usuario u</code> es de tipo <code>Pasajero</code> se crea un link entre la <code>Calificacion c</code> y la <code>Reserva r</code> relacionada <code>Usuario u</code> y al <code>Viaje vi</code> ◦ Si el <code>Usuario u</code> es de tipo <code>Conductor</code> se crea un link entre la <code>Calificacion c</code> y la <code>Reserva r</code> relacionada <code>Usuario u_calificado</code> y al <code>Viaje vi</code> ◦ Se elimina el <code>nickname</code> recordado ◦ Se elimina el <code>codigo</code> recordado

ELIMINAR VIAJE

Diagramas de Secuencia del Sistema



Contratos

Operación	<code>listarViajes() : Set (DTListarViaje)</code>
Entrada	-
Salida	Conjunto de DTListarViaje de todos los viajes
Descripción	Devuelve la información de el código, fecha, origen, destino y el nickname del conductor de todos los viajes.
Precondiciones	-
Postcondiciones	Se retorna el conjunto de los datavalue <code>dtvi</code> del tipo <code>DTListarViaje</code> correspondientes a las instancias de <code>Viaje vi</code> donde <code>dtvi.codigo = vi.codigo</code>, <code>dtvi.fecha = vi.fecha</code>, <code>dtvi.origen = vi.origen</code>, <code>dtvi.destino = vi.destino</code> y <code>dtvi.conductor = c.nickname</code> siendo <code>c</code> la instancia de <code>Conductor</code> relacionada al Vehículo <code>v</code> relacionado al <code>Viaje vi</code>

Operación	<code>detalleViaje(codigo: int): DTDetalleViaje</code>
Entrada	<code>codigo</code> : Código del viaje a consultar.
Salida	<code>DTDetalleViaje</code> del viaje consultado
Descripción	Devuelve la información detallada del viaje.
Precondiciones	Existe un Viaje <code>vi</code> con <code>vi.codigo = codigo</code>
Postcondiciones	- Se retorna un datavalue <code>dtvi</code> del tipo <code>DTDetalleViaje</code> correspondiente a la instancia de Viaje <code>vi</code>. Donde <code>dtvi.codigo = vi.codigo</code>, <code>dtvi.fecha = vi.fecha</code>, <code>dtvi.origen = vi.origen</code>, <code>dtvi.destino = vi.destino</code>, <code>dtvi.asientosPublicados = vi.asientosPublicados</code>, <code>dtvi.precio = vi.precio</code>, <code>dtvi.vehiculo = dtv</code> donde el datavalue <code>dtv</code> del tipo <code>DTDetalleVehiculo</code> corresponde a la instancia de Vehiculo <code>v</code> relacionada a la instancia de Viaje <code>vi</code> tal que <code>dtv.matricula = v.matricula</code>, <code>dtv.capacidad = v.capacidad</code>, <code>dtv.marca = v.marca</code>, <code>dtv.modelo = v.modelo</code>, <code>dtv.tipo = v.tipo</code> y <code>dtvi.reservas</code> es igual al conjunto de los datavalue <code>dtr</code> del tipo <code>DTDetalleReserva</code> de todas las instancia de Reserva <code>r</code> relacionadas a la instancia de Viaje <code>vi</code> tal que <code>dtr.asientosReservados = r.asientosReservados</code>, <code>dtr.fecha = r.fecha</code> y <code>dtr.pasajero = p.nickname</code> donde <code>p</code> es la instancia de Pasajero relacionada a la Reserva <code>r</code> - Se recuerda <code>codigo</code> pasado por parámetro

Operación	<code>eliminarViaje()</code>
Entrada	-
Salida	-
Descripción	Elimina del sistema el viaje con código junto con las instancias de Reserva y Calificación asociadas al mismo así como también los links relacionados a estos conceptos eliminados
Precondiciones	• Existe en memoria <code>codigo</code> • Existe un Viaje <code>vi</code> con <code>vi.codigo = codigo</code>
Postcondiciones	• Se elimina el link con la instancia de Vehículo <code>v</code> relacionado a la instancia de Viaje <code>vi</code> • Se eliminan los links a los Usuarios relacionados a las instancias de Calificación relacionadas a las instancias de Reservas relacionadas a la instancia de Viaje <code>vi</code> • Se eliminan las instancias de Calificación y sus links relacionadas a las instancias de Reservas relacionadas a la instancia de Viaje <code>vi</code> • Se eliminan el link al Pasajero de las instancias de Reservas relacionadas a la instancia del Viaje <code>vi</code> • Se eliminan las instancias de Reservas y sus links relacionadas a la instancia del Viaje <code>vi</code> • Se elimina el link a Vehículo relacionado a la instancia del Viaje <code>vi</code> • Se elimina la instancia de Viaje <code>vi</code> • Se libera la memoria de <code>codigo</code>

Operación	<code>cancelarEliminarViaje()</code>
Entrada	-
Salida	-
Descripción	Libera la memoria del caso de uso
Precondiciones	• Existe en memoria <code>codigo</code>
Postcondiciones	• Se libera la memoria de <code>codigo</code>

DATA TYPES Y ENUMERADOS UTILIZADOS

<<datatype>> DTVehiculosConductor - matricula: String - modelo: String - capacidad: int	<<datatype>> DTConsultaViaje - codigo: int - marca: String - modelo: String - conductor: String - calificacionProm: float - precioTotal: float	<<datatype>> DTUsuario - nickname: String - nombre: String	<<datatype>> DTUsuarioViaje - nickname: String - tipo: TipoUsuario	«enumeration» TipoUsuario Conductor Pasajero
<<datatype>> DTListarViaje - codigo: int - fecha: DTFecha - origen: String - destino: String - conductor: String	<<datatype>> DTDetalleViaje - codigo: int - fecha: DTFecha - origen: String - destino: String - asientosPublicados: int - precio: float - vehiculo: DTDetalleVehiculo - reservas: Set(DTDetalleReserva)	<<datatype>> DTDetalleReserva - asientosResevados: int - fecha: DTFecha - pasajero: String	<<datatype>> DTDetalleVehiculo - matricula: String - capacidad: int - marca: String - modelo: String - tipo: TipoVehiculo	